

TRAINEE LAB GUIDE

Shopify Agentic AI Developer Bootcamp — Koenig Solutions for V Group Inc.

Lab 2.3

Scaffold an Agent Project (Claude Agent SDK)

Module	2 — Building Agents with the Claude API & Agent SDK
Topic	2.3 — Claude Agent SDK Essentials
Estimated time	30–40 minutes
Tools needed	Terminal, Node.js 20+, a code editor, an active Claude API key
Prerequisite	Lab 2.2 complete; Topic 2.3 briefing (agent loop, <code>createSdkMcpServer</code> , permission modes) on the program website

What You're Building

GOAL

A Node.js/TypeScript project scaffolded with the **Claude Agent SDK** — one custom tool exposed via `createSdkMcpServer`, an explicitly chosen permission mode, and confirmation that a natural-language prompt actually triggers the tool.

This replaces the hand-rolled request → tool-call → result loop from Lab 2.2 with the SDK's managed agent runtime — same idea, less plumbing.

Steps 1–3: Set Up the Project

1 Create a project folder

In a terminal, outside of any other project:

TERMINAL

```
mkdir agent-scaffold
cd agent-scaffold
npm init -y
```

What you'll see: a new folder containing a `package.json` file.

2 Store your API key safely

Create a file named `.env` in the project folder with your Anthropic API key:

.ENV

```
ANTHROPIC_API_KEY=sk-ant-your-key-here
```

Before you go further: create a `.gitignore` file containing just the line `.env` — this key should never end up committed to git, even in a lab exercise.

3 Install TypeScript tooling and the Agent SDK

TERMINAL

```
npm install @anthropic-ai/claude-agent-sdk zod dotenv
npm install -D typescript ts-node @types/node
npx tsc --init
```

Open the generated `tsconfig.json` and set `"module": "NodeNext"` and `"moduleResolution": "NodeNext"` so TypeScript can resolve the SDK's ESM exports correctly.

What you'll see: `@anthropic-ai/claude-agent-sdk`, `zod` and `dotenv` under `dependencies`; `typescript`, `ts-node` and `@types/node` under `devDependencies`; a new `tsconfig.json`.

Steps 4–5: Scaffold the Agent

4 Open Claude Code and describe the task

In your project folder, run `claude` to start Claude Code, then give it this spec:

PROMPT TO CLAUDE CODE

```
Build a Node.js/TypeScript project (entry file agent.ts) using the
@anthropic-ai/claude-agent-sdk package:
```

1. Loads ANTHROPIC_API_KEY from a .env file via dotenv
2. Defines one custom tool via createSdkMcpServer, named "inventory-tools", with a single tool get_stock_level that takes { sku: string } (validated with zod) and returns a mock stock count as text, e.g. "SKU WB-1L: 42 in stock"
3. Runs the agent's query loop with that MCP server registered, sending the prompt "How many of SKU WB-1L do we have in stock?"
4. Sets an EXPLICIT permission mode that requires approval before any tool call runs – do not leave it on an unreviewed default, and add a one-line comment explaining what the chosen mode does
5. Prints every turn of the agent's response to the console, including when the tool is called and what it returned – not just the final answer

Use TypeScript throughout with proper types, no `any`.

5 Review the diff before accepting

Claude Code will propose creating `agent.ts` and updating `package.json`. Before accepting, check:

Check	Why it matters
Does the MCP server register the tool via <code>createSdkMcpServer + tool()</code> , with a <code>zod</code> input schema?	This is the SDK-native way to expose a tool, not a raw <code>input_schema</code> object like Lab 2.2
Does the tool's handler return the <code>{ content: [{ type: "text", text: ... }] }</code> shape the SDK expects?	A mismatched return shape fails silently or throws deep inside the SDK
Is the permission mode set explicitly and visibly in the code, not left implicit?	The whole point of this step — a default you haven't checked is a guardrail you don't actually have
Does the script print intermediate tool-call turns, not just the final answer?	You need to actually see the round trip to verify the tool fired

Does the project compile with `tsc` / run with `ts-node` without type errors?

Type errors here usually mean a module-resolution mismatch, not a logic bug

If anything's missing, don't fix it yourself — ask Claude Code directly: "set the permission mode explicitly and add a comment explaining it" or "print the `tool_use` and `tool_result` turns, not just the final message." That request-and-iterate step is the point of the exercise.

Steps 6–7: Run and Verify

6 Compile and run it

TERMINAL

```
npx tsc --noEmit
npx ts-node agent.ts
```

What you'll see: `tsc --noEmit` reports no type errors, then the console shows the agent recognizing the stock-lookup request, calling `get_stock_level` with `sku: "WB-1L"`, and — depending on your permission mode — either pausing for approval or proceeding straight to a final answer reporting the mock stock count.

7 Confirm the acceptance checklist

- ☐ Project runs with `tsc` / `ts-node` without type errors
- ☐ At least one custom tool is registered via `createSdkMcpServer`
- ☐ A natural-language prompt that should trigger the tool actually does
- ☐ Permission mode is set deliberately, not left on a default you haven't checked

Troubleshooting

Symptom	Likely cause	Try this
Type errors importing the Agent SDK or <code>zod</code>	<code>tsconfig.json</code> module resolution not set for ESM packages	Confirm <code>"module"</code> and <code>"moduleResolution"</code> are both set to <code>"NodeNext"</code>
Agent never calls the tool	Tool description too vague, or the MCP server isn't actually registered on the agent call	Confirm the server is passed into the agent's tools/servers list, and sharpen the tool's description
Script pauses with no way to respond	Permission mode requires confirmation but no approval handler is wired up	For this lab, either choose a mode that auto-approves for local testing, or add the interactive approval hook the SDK docs describe
Cannot find module '@anthropic-ai/claude-agent-sdk'	Package not installed, or installed in the wrong project folder	Re-run <code>npm install</code> in this folder; confirm it's listed in <code>package.json</code>

`ANTHROPIC_API_KEY` is
undefined

`.env` not loaded, or in the wrong
folder

Confirm `dotenv.config()` runs before the agent
is created

Where This Goes Next

This scaffold — one MCP server, one tool, an explicitly chosen permission mode — is the exact shape Lab 2.6's multi-tool agent extends with three tools and real conversation memory. It's also what Module 7 later exposes over the network as a standards-compliant MCP server any MCP-aware client can use.

Shopify Agentic AI Developer Bootcamp — Koenig Solutions for V Group Inc. · Module 2: Building Agents with the Claude API & Agent SDK · Full module content: see the program website, Module 2 page.